

```
# Importar el dataset
from google.colab import files
uploaded = files.upload()
```

[Elegir archivos](#) Womens Cl... Reviews.csv

Womens Clothing E-Commerce Reviews.csv(text/csv) - 8483448 bytes, last modified: 27/9/2019 - 100% done
Saving Womens Clothing E-Commerce Reviews.csv to Womens Clothing E-Commerce Reviews (1).csv

```
# convertir el dataset a pandas
import pandas as pd
df = pd.read_csv("Womens Clothing E-Commerce Reviews.csv")

# Ver 5 primeras filas
df.head()
```

	Unnamed: 0	Clothing ID	Age	Title	Review Text	Rating	Recommended IND	Positive Feedback Count	Division Name	Department Name	Class Name
0	0	767	33	NaN	Absolutely wonderful - silky and sexy and comf...	4	1	0	Initmates	Intimate	Intimates
1	1	1080	34	NaN	Love this dress! it's sooo pretty. i happene...	5	1	4	General	Dresses	Dresses
2	2	1077	60	Some major design flaws	I had such high hopes for this dress and reall...	3	0	0	General	Dresses	Dresses

Próximos pasos: [Generar código con df](#) [New interactive sheet](#)

```
# Entender el dataset
print("Shape:", df.shape)
print("\nInfo:")
df.info()
```

Shape: (23486, 11)

Info:

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 23486 entries, 0 to 23485
```

Data columns (total 11 columns):

#	Column	Non-Null Count	Dtype
0	Unnamed: 0	23486 non-null	int64
1	Clothing ID	23486 non-null	int64
2	Age	23486 non-null	int64
3	Title	19676 non-null	object
4	Review Text	22641 non-null	object
5	Rating	23486 non-null	int64
6	Recommended IND	23486 non-null	int64
7	Positive Feedback Count	23486 non-null	int64
8	Division Name	23472 non-null	object
9	Department Name	23472 non-null	object
10	Class Name	23472 non-null	object

```
dtypes: int64(6), object(5)
memory usage: 2.0+ MB
```

```
#eliminar filas sin texto
df = df.dropna(subset=["Review Text"])
print("Shape después de limpiar:", df.shape)
```

Shape después de limpiar: (22641, 11)

```
# creacion variable objetivo
def label_y(rating):
    if rating >= 4:
        return 1
    return 0

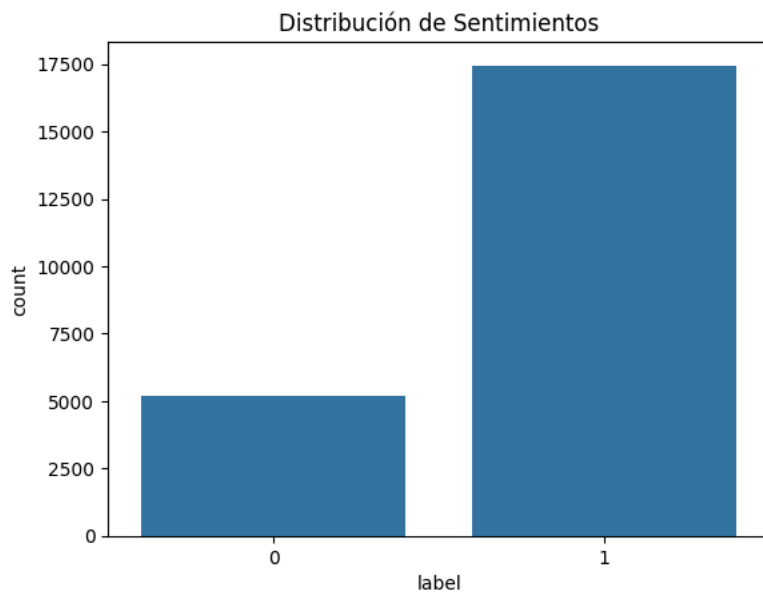
df["label"] = df["Rating"].apply(label_y)
```

```
df[["Rating", "label"]].head()
```

	Rating	label	
0	4	1	
1	5	1	
2	3	0	
3	5	1	
4	5	1	

```
# Visualización para entender los datos
import seaborn as sns
import matplotlib.pyplot as plt

sns.countplot(data=df, x="label")
plt.title("Distribución de Sentimientos")
plt.show()
```



```
df = df[["Review Text", "label"]]
df.head()
```

	Review Text	label	
0	Absolutely wonderful - silky and sexy and comf...	1	
1	Love this dress! it's sooo pretty. i happene...	1	
2	I had such high hopes for this dress and reall...	0	
3	I love, love, love this jumpsuit. it's fun, fl...	1	
4	This shirt is very flattering to all due to th...	1	

Próximos pasos: [Generar código con df](#) [New interactive sheet](#)

```
!pip install transformers torch accelerate -q
```

```
import numpy as np
import time

from sklearn.model_selection import train_test_split
from sklearn.metrics import accuracy_score, precision_score, recall_score, f1_score, classification_report
from transformers import pipeline
```

```
# division de datos en entrenamiento y prueba
X = df["Review Text"].astype(str)
```

```

y = df["label"]
X_train, X_test, y_train, y_test = train_test_split(
    X,
    y,
    test_size=0.2,
    random_state=42,
    stratify=y
)

```

```

X_eval = X_test[:500].tolist()
y_eval = y_test[:500].tolist()

```

```

def evaluar_modelo(nombre_modelo, modelo_hf, tipo_salida="positive_negative"):

    print(f"Evaluando: {nombre_modelo}")

    clasificador = pipeline(
        "sentiment-analysis",
        model=modelo_hf,
        truncation=True
    )

    predicciones = []
    inicio = time.time()

    for texto in X_eval:
        resultado = clasificador(texto)[0]
        etiqueta = resultado["label"]

        if tipo_salida == "positive_negative":
            if etiqueta.upper() == "POSITIVE":
                predicciones.append(1)
            else:
                predicciones.append(0)

        elif tipo_salida == "stars":
            estrellas = int(etiqueta[0])

            if estrellas >= 4:
                predicciones.append(1)
            else:
                predicciones.append(0)

    fin = time.time()

    return {
        "Modelo": nombre_modelo,
        "Accuracy": round(accuracy_score(y_eval, predicciones), 4),
        "Precision": round(precision_score(y_eval, predicciones), 4),
        "Recall": round(recall_score(y_eval, predicciones), 4),
        "F1-score": round(f1_score(y_eval, predicciones), 4),
        "Tiempo_segundos": round(fin - inicio, 2)
    }

```

```

resultados = []

resultados.append(
    evaluar_modelo(
        "DistilBERT SST-2",
        "distilbert-base-uncased-finetuned-sst-2-english",
        "positive_negative"
    )
)

resultados.append(
    evaluar_modelo(
        "RoBERTa Twitter Sentiment",
        "cardiffnlp/twitter-roberta-base-sentiment-latest",
        "positive_negative"
    )
)

resultados.append(
    evaluar_modelo(

```

```
"BERT Multilingual Stars",
"nlptown/bert-base-multilingual-uncased-sentiment",
"stars"

)
)
```

Evaluando: DistilBERT SST-2
Loading weights: 100% 104/104 [00:00<00:00, 3.46it/s]
Evaluando: RoBERTa Twitter Sentiment
Loading weights: 100% 201/201 [00:00<00:00, 5158.54it/s]
[transformers] RobertaForSequenceClassification LOAD REPORT from: cardiffnlp/twitter-roberta-base-sentiment-latest
Key | Status | |
-----+-----+--
roberta.pooler.dense.bias | UNEXPECTED | |
roberta.pooler.dense.weight | UNEXPECTED | |

Notes:
- UNEXPECTED: can be ignored when loading from different task/architecture; not ok if you expect identical arch.

Evaluando: BERT Multilingual Stars
Loading weights: 100% 201/201 [00:00<00:00, 3188.11it/s]

```
tabla_resultados = pd.DataFrame(resultados)

tabla_resultados = tabla_resultados.sort_values(
    by="F1-score",
    ascending=False
)

tabla_resultados
```

	Modelo	Accuracy	Precision	Recall	F1-score	Tiempo_segundos	
2	BERT Multilingual Stars	0.876	0.9645	0.8670	0.9132	246.81	
0	DistilBERT SST-2	0.860	0.9158	0.8963	0.9059	127.40	
1	RoBERTa Twitter Sentiment	0.840	0.8834	0.9069	0.8950	194.54	

Próximos pasos: [Generar código con tabla_resultados](#) [New interactive sheet](#)

```
mejor_modelo = tabla_resultados.iloc[0]
mejor_modelo
```

	2
Modelo	BERT Multilingual Stars
Accuracy	0.876
Precision	0.9645
Recall	0.867
F1-score	0.9132
Tiempo_segundos	246.81

dtype: object

```
nombre_modelo_ganador = tabla_resultados.iloc[0]["Modelo"]

print(nombre_modelo_ganador)

BERT Multilingual Stars
```

```
modelo_final = pipeline(
    "sentiment-analysis",
    model="nlptown/bert-base-multilingual-uncased-sentiment"
)

Loading weights: 100% 201/201 [00:00<00:00, 2478.18it/s]
```

```
nuevas_resenas = [
    "This dress is amazing and fits perfectly",
```

```
"Terrible quality and very uncomfortable"
]

for texto in nuevas_resenas:
    resultado = modelo_final(texto)[0]
    estrellas = int(resultado["label"][0])

    sentimiento = 1 if estrellas >= 4 else 0
    etiqueta = "Positivo" if sentimiento == 1 else "Negativo"

    print("Texto:", texto)
    print("Predicción:", etiqueta)
    print("Estrellas:", resultado["label"])
    print("Confianza:", round(resultado["score"], 4))
    print()
```

Texto: This dress is amazing and fits perfectly
Predicción: Positivo
Estrellas: 5 stars
Confianza: 0.9116

Texto: Terrible quality and very uncomfortable
Predicción: Negativo
Estrellas: 1 star
Confianza: 0.8883